PLATEFORME D'OBSERVABILITÉ INTELLIGENTE

Cassandra

Intelligent Observability and
Automated Incident Detection Platform

Rapport de stage d'été

Stage d'été — 08/06/2026 - 07/08/2026

Chanaz Jammoussi
Talan Tunisie
ISIMM - Institut Supérieur d'Informatique et de Mathématiques de Monastir
Encadrant entreprise : Mme Ines Boukhris
Encadrant académique : Mr Imed Abbessi

Année universitaire 2025 - 2026

https://github.com/ChanazJammoussi/api-performance-platform-Cassandra-

Sommaire

Résumé 4

Abstract 4

Remerciements 6

1. Présentation de l'organisme d'accueil 7

2. Contexte du stage et objectifs 9

3. Méthodologie et organisation du projet 11

4. Problématique technique et enjeux 13

5. Objectifs techniques et périmètre 14

6. Architecture générale 15

7. Ingestion et pipeline de données 17

8. Génération de scénarios de panne et fault injection 18

9. Features et baseline saisonnière 21

10. Architecture de détection multicouche 23

11. Cycle de vie du modèle ML 25

12. Machine à états d'alerte 26

13. Corrélation déploiement et graphe de services 27

14. Génération d'explications d'incidents assistée par LLM 29

15. Méthodologie d'évaluation 31

16. Résultats mesurés 32

17. Sécurité et self-observabilité 35

18. Limites et travaux futurs 36

19. Conclusion 37

Références 38

Résumé

Contexte

Les plateformes applicatives modernes sont distribuées et évoluent en continu. Les approches traditionnelles d'observabilité restent encore largement fondées sur des **seuils statiques** : une alerte se déclenche quand une métrique dépasse une valeur fixe. Ce modèle ignore la **saisonnalité** du trafic (heures de pointe, jours ouvrés), génère du bruit, et détecte tard les dégradations progressives. Il n'explique ni la **cause** ni l'**action à mener**.

Problème

Détecter une dégradation de performance d'API *plus tôt* et avec *moins de faux positifs* qu'un seuillage statique, tout en **attribuant** l'incident à un déploiement probable et en l'**expliquant** en langage naturel — tout en évitant une dépendance à des jeux de données historiques annotés, généralement difficiles à obtenir dans les environnements industriels.

Solution

Cassandra apprend la signature normale de chaque endpoint via une **baseline saisonnière** conditionnée (jour x heure), puis combine cette déviation avec une **détection d'anomalies non supervisée** (Isolation Forest) sur des features *endpoint-relatives*. Un moteur de **corrélation** relie chaque dégradation aux déploiements récents ; une couche **LLM** produit un résumé d'incident actionnable envoyée vers Slack. Chaque faute injectée dans le banc d'évaluation est un label de vérité-terrain, ce qui rend la performance **directement mesurable**.

Résultats

Une campagne historique comprenant 31 fenêtres d'injection valides a été utilisée pour l'évaluation. Le détecteur multicouche atteint un taux de détection de **71 %**, contre 65 % pour une approche basée uniquement sur des seuils statiques. L'amélioration atteint **+20 points** pour les scénarios d'échelon de latence (latency_step), avec une augmentation limitée du taux de faux positifs (+0,16 FP/h). Une évaluation qualitative des explications générées par le LLM montre une bonne pertinence des résumés d'incidents. Un comparatif supervisé (gradient boosting) obtient une PR-AUC quasi identique, ce qui **justifie le choix non supervisé**.

La plateforme est entièrement reproductible via Docker Compose et validée par une suite de 105 tests automatisés couvrant les principaux composants développés.

Abstract

Cassandra is an observability platform that **detects**, **attributes** and **explains** API endpoint performance degradation. Rather than static threshold alerting, it learns the normal behavioral signature of each endpoint — a **seasonal baseline** conditioned on day-of-week and hour — and combines it with **unsupervised anomaly detection** (Isolation Forest) over endpoint-relative engineered features. A correlation engine links each degradation to recent deployment events, and an **LLM explanation layer** generates a plain-language incident summary delivered to Slack: what degraded, by how much versus the expected baseline, which deploy is suspected, and what to check first.

Every injected fault is a **ground-truth label**, making detection rate, false positive rate and detection delay directly measurable against a static-threshold baseline. The layered detector reaches **71 %** detection versus 65 % for static thresholding (**+20 points on gradual latency-step faults**) at a modest false positive cost. The entire stack is reproducible with a single `docker compose up` and validated by 105 automated tests with continuous integration.

Remerciements

À l'issue de ce stage, je tiens à exprimer ma sincère gratitude à l'ensemble des personnes qui ont contribué, de près ou de loin, à la réussite de cette expérience professionnelle.

Je remercie chaleureusement **Talan Tunisie** pour son accueil, ainsi que pour la confiance qui m'a été accordée en me confiant un sujet à la fois stimulant et enrichissant.

J'adresse mes remerciements les plus sincères à mon encadrante en entreprise, **Mme Inès Boukhris**, pour sa disponibilité, la pertinence de ses conseils et son accompagnement attentif tout au long de ce projet.

Je remercie également mon encadrant académique, **M. Imed Abbessi**, pour son suivi, ses orientations et la rigueur apportée dans l'accompagnement de ce travail.

Enfin, j'exprime ma reconnaissance à l'ensemble de l'équipe pour son esprit d'entraide et son soutien, ainsi qu'au corps enseignant de l'**ISIMM** pour la qualité de la formation dispensée.

1 | Présentation de l'organisme d'accueil

Ce chapitre présente l'entreprise d'accueil, son implantation en Tunisie, ses domaines d'expertise et le service au sein duquel s'est déroulé le stage.

1.1 Le groupe Talan

Fondé en 2002 par Mehdi Houas, Éric Benamou et Philippe Cassoulat, **Talan** est un groupe international de conseil spécialisé dans l'**innovation** et la **transformation des entreprises par la technologie**.

Le groupe accompagne ses clients dans leurs projets de transformation numérique en s'appuyant sur plusieurs domaines d'expertise, notamment la data, l'intelligence artificielle, le cloud, l'ingénierie logicielle et les technologies émergentes.

Présent dans 21 pays sur les cinq continents, Talan regroupe aujourd'hui plus de **6 000 collaborateurs**. Son approche repose sur la combinaison de l'expertise technologique, de l'innovation et de l'accompagnement des organisations dans leurs évolutions numériques.

1.2 Talan en Tunisie

Talan Tunisie constitue un centre d'expertise et de delivery du groupe, participant au développement de solutions technologiques pour des clients nationaux et internationaux. Les équipes tunisiennes interviennent notamment dans les domaines de la data, de l'intelligence artificielle, du développement logiciel et des technologies cloud.

Grâce à ses compétences techniques et à son intégration dans l'écosystème international du groupe, **Talan Tunisie** contribue à la réalisation de projets innovants combinant expertise métier et nouvelles technologies. Le stage présenté dans ce rapport s'inscrit dans cette dynamique à travers le développement de la plateforme **Cassandra** dédiée à l'observabilité intelligente et à la détection d'incidents.

1.3 Domaines d'expertise

Domaine	Périmètre
Data & Intelligence Artificielle	Valorisation de la donnée, machine learning, MLOps, observabilité
Cloud & DevOps	Migration cloud, conteneurisation, automatisation, CI/CD
Ingénierie logicielle	Conception et développement d'applications et de plateformes
Transformation digitale	Conseil, cadrage et conduite du changement
Cybersécurité	Sécurisation des systèmes et des données
Technologies émergentes	Innovation appliquée (blockchain, IoT, GenAI)

1.4 Cadre du stage et positionnement du projet

Le stage a été réalisé au sein de **Talan Tunisie** sous l'encadrement de **Mme Inès Boukhris**. Le projet Cassandra s'inscrit dans un environnement technologique combinant plusieurs domaines d'expertise,

notamment l'ingénierie logicielle, l'observabilité des systèmes distribués et l'intelligence artificielle.

Ce positionnement correspond aux objectifs du projet, qui consiste à concevoir une plateforme intelligente capable de collecter et d'analyser des métriques applicatives, de détecter des dégradations de performance et d'assister l'analyse des incidents à l'aide de techniques d'apprentissage automatique.

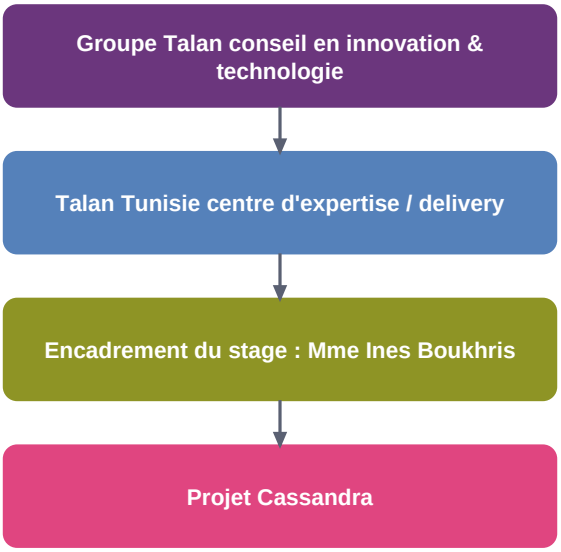


Figure 1 — Organigramme simplifié : positionnement du projet Cassandra dans l'organisation d'accueil.

2 Contexte du stage et objectifs

Ce chapitre présente le cadre général du stage, le sujet confié, la problématique ayant motivé le projet, ainsi que les objectifs techniques et les livrables associés.

2.1 Cadre du stage

Le présent travail a été réalisé dans le cadre d'un **stage d'ingénieur** d'une durée de deux mois, du **08/06/2026 au 07/08/2026**, au sein de **Talan Tunisie**. Ce stage s'inscrit dans le cursus de formation d'ingénieur de l'**ISIMM** et vise à mettre en application les connaissances acquises à travers la réalisation d'un projet technologique en environnement professionnel.

2.2 Sujet et problématique métier

Les applications distribuées modernes génèrent un volume important de métriques, et les incidents de performance ont un coût direct (expérience utilisateur dégradée) et indirect (temps d'investigation). La supervision traditionnelle par **seuils statiques** produit trop d'alertes peu exploitables, détecte tardivement les dérives progressives et n'apporte ni cause ni piste d'action.

Le sujet confié consiste à **concevoir et réaliser Cassandra**, une plateforme d'observabilité capable de **détecter**, d'**attribuer** et d'**expliquer** les dégradations de performance des API, et de démontrer son apport de manière **mesurable** face au seuillage statique.

2.3 Objectifs du stage

Catégorie	Objectifs
Objectifs généraux	Concevoir une plateforme d'observabilité de bout en bout, détectant et expliquant les dégradations de performance des API.
Objectifs spécifiques	Baseline saisonnière, détection en couches (static / baseline / Isolation Forest), corrélation déploiement, explication LLM, évaluation chiffrée par fault injection.
Objectifs personnels	Développer les compétences en observabilité, en apprentissage automatique supervisé et non supervisé, ainsi qu'en pratiques modernes d'ingénierie logicielle (Docker, tests, CI/CD).

2.4 Livrables attendus

- Une plateforme **reproductible** déployable via Docker Compose.
- Une **évaluation quantitative** comparative entre l'approche proposée et un seuillage statique sur un banc d'injection de fautes.
- Des **tableaux de bord Grafana** dédiés au suivi opérationnel, à l'évaluation et à la self-observabilité.
- Une **documentation technique** complète ainsi que le présent rapport.

POINT CLE

Enjeu pour l'entreprise : disposer d'un socle d'observabilité intelligente, interprétable et reproductible, transférable à des services réels via OpenTelemetry, sans instrumentation propriétaire.

3 Méthodologie et organisation du projet

Ce chapitre présente la méthodologie adoptée, le planning prévisionnel, ainsi que l'organisation des différentes phases de développement de la plateforme Cassandra.

3.1 Méthodologie adoptée

Le projet a été conduit selon une approche **itérative et incrémentale**, inspirée des pratiques Agile. Le développement a été organisé en plusieurs phases successives, chacune validée par un livrable fonctionnel. Cette démarche a permis de construire progressivement la plateforme Cassandra et de réduire les risques liés à l'intégration des différents composants.

3.2 Planning prévisionnel

Le projet a été découpé en six phases sur environ 52 jours ouvrés, du banc de démo jusqu'à la campagne d'évaluation finale :

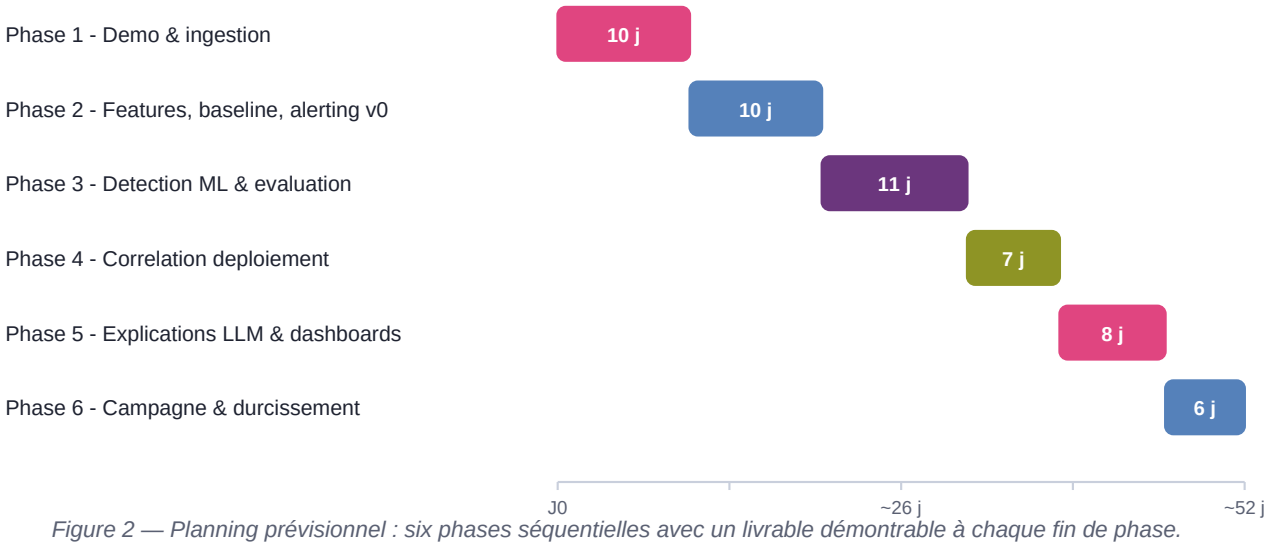


Figure 2 — Planning prévisionnel : six phases séquentielles avec un livrable démontrable à chaque fin de phase.

3.3 Organisation en phases

Phase	Durée	Livrable / checkpoint
1 — Démonstrateur et ingestion	10 j	Métriques RED visibles pour les scénarios injectés
2 — Features, baseline et alerting	10 j	Première détection avec génération d'alertes
3 — Détection ML et évaluation	11 j	Comparaison layered vs seuils statiques
4 — Corrélation déploiement	7 j	Identification du déploiement associé
5 — Explication LLM et dashboards	8 j	Démonstration complète du cycle d'incident
6 — Campagne finale et durcissement	6 j	Résultats consolidés et environnement reproductible

3.4 Environnement et outils de travail

Catégorie	Outils / technologies
Langage & frameworks	Python, FastAPI
ML & data	scikit-learn (Isolation Forest), numpy, pandas
Observabilité	OpenTelemetry, Prometheus, Grafana
Stockage	TimescaleDB (PostgreSQL)
Charge & tests	k6, pytest
Conteneurisation	Docker, Docker Compose
Versionning & CI	Git, GitHub, GitHub Actions
LLM & notification	Gemini (google-genai), Slack (Block Kit)

3.5 Pratiques d'ingénierie et suivi

- **Gestion de versions et CI :** utilisation de Git/GitHub et d'un workflow GitHub Actions pour automatiser les vérifications.
- **Tests automatisés :** une suite de tests couvre les composants principaux du service de détection afin de limiter les régressions.
- **Reproductibilité :** déploiement de l'ensemble de la plateforme via Docker Compose.
- **Documentation et suivi :** maintien de la documentation technique et suivi régulier avec l'encadrante en entreprise.

4 | Problématique technique et enjeux

Ce chapitre présente les limites des approches traditionnelles de supervision et la problématique à laquelle Cassandra cherche à répondre.

4.1 Un paysage applicatif distribué et mouvant

Les applications modernes reposent sur des architectures distribuées dont le comportement varie selon les services, les périodes de trafic et les évolutions du système. Les dégradations de performance peuvent être causées par un déploiement, une saturation de ressources ou une propagation d'incident entre services.

4.2 Les limites du seuillage statique

La supervision basée sur des seuils fixes présente plusieurs limites :

- **Sensibilité limitée à la saisonnalité** : un seuil unique ne reflète pas les variations normales du trafic.
- **Détection tardive** : les dégradations progressives peuvent être détectées après l'apparition de l'impact utilisateur.
- **Manque de contexte** : une alerte indique un symptôme sans fournir d'explication ni de cause probable.

> **DÉFINITION — BASELINE SAISONNIÈRE**
>
> Une **baseline saisonnière** modélise la valeur *attendue* d'une métrique pour un endpoint donné à une heure et un jour donnés, sous forme d'une bande de quantiles (p10–p90). Une observation est anormale non pas dans l'absolu, mais *relativement* à cette bande.

4.3 Problématique retenue

La thèse du projet est **relative** et mesurable : détecter *plus* et *plus tôt* que le seuillage statique, avec un compromis maîtrisé sur le taux de faux positifs, puis **attribuer** et **expliquer** l'incident. En l'absence de données historiques d'incidents, l'approche principale repose sur la **détection non supervisée**, complétée par une comparaison avec une approche supervisée afin d'évaluer sa pertinence.

5 Objectifs techniques et périmètre

5.1 Objectifs

- Détecter les dégradations de performance par endpoint avec une **approche adaptative** et évaluer son apport face au seuillage statique.
- **Attribuer** chaque dégradation à une cause probable au niveau déploiement, via corrélation temporelle, avec un **score d'imputation dans [0, 1]** basé sur la proximité temporelle (corrélation, non causalité certaine).
- Générer des explications d'incident **actionnables** avec un LLM, déclenchées uniquement sur alerte confirmée (maîtrise du coût).
- S'intégrer à une chaîne d'observabilité standard basée sur **OpenTelemetry**.
- Mettre en place une **évaluation reproductible** à partir de scénarios de fautes contrôlées.

5.2 Hors périmètre

Afin de conserver un périmètre adapté à la durée du stage, les éléments suivants ne sont pas couverts :

- Prédiction garantie des incidents ou remplacement d'un système APM complet.
- Remédiation automatique des incidents.
- Analyse détaillée du code source ou profilage bas niveau.
- Gestion multi-tenant et mise à l'échelle horizontale avancée.
- Développement d'une interface dédiée, les visualisations étant réalisées avec Grafana.

6 | Architecture générale

La plateforme est un pipeline à sept étapes, du trafic synthétique jusqu'à la notification expliquée, entièrement orchestré par Docker Compose.

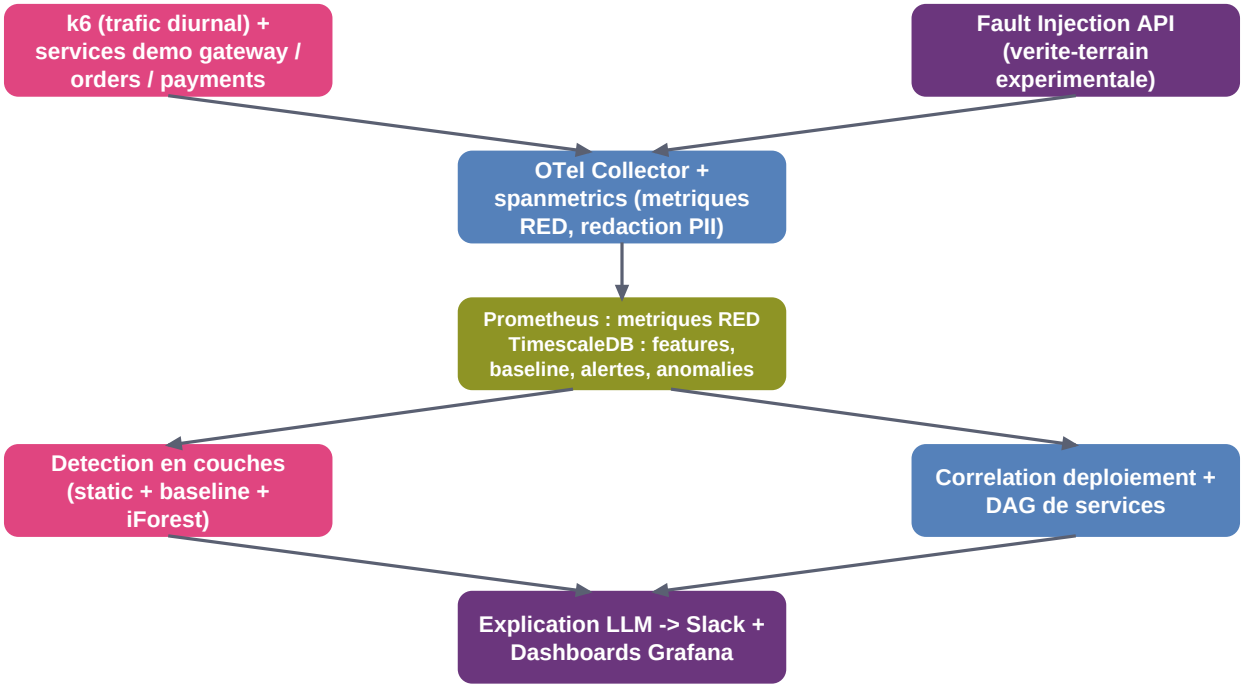


Figure 3 — Pipeline end-to-end : génération de trafic, ingestion, features/baseline, détection, corrélation, explication, notification.

6.1 Les sept étapes

Ces étapes correspondent aux principaux composants fonctionnels de la plateforme, de la collecte jusqu'à l'explication de l'incident. Chaque étape correspond à un composant fonctionnel identifiable et peut être validée séparément, ce qui permet une validation incrémentale :

#	Étape	Rôle
1	Génération de trafic	Microservices démo sous charge k6 diurnal, avec API de fault injection (dev/eval).
2	Ingestion	OTel Collector + connecteur spanmetrics : métriques RED sur tout le trafic.
3	Features & baseline	Fenêtres par endpoint (p50/p95/p99, 4xx/5xx, rps) + baseline saisonnière.
4	Détection	Détecteur en couches (static + baseline + Isolation Forest) -> score d'anomalie.
5	Corrélation	Rapprochement des dégradations avec les déploiements dans une fenêtre temporelle de corrélation.
6	Explication	Un LLM assemble le contexte en un résumé d'incident lisible.

#	Étape	Rôle
7	Notification	Alertes structurées vers Slack ; visualisation de l'état et de l'historique via Grafana.

6.2 Services et ports

Service	Port (hôte)	Rôle
gateway	8000	Point d'entrée, proxy vers orders / payments
orders	8001	Service commandes — instrumenté OTel, API fault interne
payments	8002	Service paiements — instrumenté OTel, API fault interne
otel-collector	4317	Récepteur OTLP gRPC — ingestion traces/métriques depuis les services
	4318	Récepteur OTLP HTTP — alternative HTTP/protobuf au gRPC
	8888	Exposition des métriques internes du collector (scraping Prometheus)
prometheus	9090	Stockage des métriques (remote-write)
timescaledb	5434 (hôte)	Port hôte — accès depuis l'hôte et les outils externes
	5432 (conteneur)	Port PostgreSQL standard dans le conteneur
detector	—	Détection en couches, state machine, corrélation, LLM
trainer	—	Réentraînement nightly de l'Isolation Forest
deploy-api	8090	Registre des déploiements (POST/GET /deploys)
grafana	3000	Dashboards santé / évaluation / self-observabilité

CHOIX TECHNIQUE

La détection tourne sur une **cadence d'une minute**. Le délai entre l'événement et le message Slack est **de l'ordre de 3 minutes** dans le détecteur live : 3 cycles de 60 s sont nécessaires avant FIRING (`PENDING_WINDOWS=2`, compteur démarre à 0). Une règle de *watermark* exclut la dernière fenêtre incomplète (ne jamais agir sur des données partielles).

7 | Ingestion et pipeline de données

Le Collector OpenTelemetry reçoit les spans OTLP. Le connecteur `spanmetrics` génère les métriques RED (Rate, Errors, Duration), exploitées par le pipeline de détection. Aucun mécanisme d'échantillonnage n'est configuré dans cette chaîne d'ingestion.

7.1 Deux décisions résolues en amont

Stratégie d'histogramme. Les percentiles issus de buckets grossiers sont trop bruités pour en dériver des features fiables. Des buckets explicites fins (16 paliers de 2 ms à 15 s) sont donc utilisés : c'est une **exigence de correction**, pas du tuning.

Contrôle de cardinalité. `endpoint_id = méthode + http.route` (templateée), ce qui évite l'explosion par URL brute. Le scraper n'interroge qu'une liste fixe de six endpoints déclarés explicitement : seules ces séries sont écrites en base, les autres métriques Prometheus sont ignorées.

7.2 Rédaction PII et absence de bus de messages

Un filtre d'attributs supprime par denylist les attributs de span porteurs de PII connus (url, ip, identifiant utilisateur) avant export — aucune donnée de payload n'est ingérée. Le Collector exporte les métriques RED vers **Prometheus** (`prometheusremotewrite`) ; le scraper lit ensuite Prometheus et alimente TimescaleDB. **Aucun bus de messages** n'intervient dans cette chaîne.

> **DÉFINITION — MÉTRIQUES RED**
>
> Les métriques RED (*Rate, Errors, Duration*) résument la santé d'un service : débit de requêtes, taux d'erreurs, distribution de latence. C'est le socle minimal, standard et non propriétaire, sur lequel toute la détection s'appuie.

8 | Génération de scénarios de panne et fault injection

Le banc de démonstration n'est pas un artefact secondaire : c'est la source des scénarios de faute contrôlés servant de vérité-terrain expérimentale (*ground truth*) et rendant l'évaluation quantitative possible.

8.1 Microservices de démonstration

Trois services (`gateway`, `orders`, `payments`) forment une topologie d'appels réaliste (`gateway -> orders`, `gateway -> payments -> orders`), chacun instrumenté avec **OpenTelemetry** pour la collecte des données d'observabilité, avec des endpoints regroupés par modèle d'URL afin de faciliter l'analyse des métriques. Un pool de connexions PostgreSQL rend la saturation réaliste.

8.2 Charge et saisonnalité

Le générateur k6 produit un trafic en forme de journée compressée (une « journée » de saisonnalité rejouée en 2 heures, avec une variation de 5 à 50 VUs) afin de créer des variations temporelles réalistes permettant à la baseline d'apprendre le comportement normal du système. Les clés de baseline (`dow`, `hour_bucket`) sont définies indépendamment de la durée de la simulation et restent donc réutilisables lors d'exécutions ultérieures.

Stage	Durée	VUs cible	Période simulée
1	20 min	25	Matin — montée
2	40 min	50	Midi — pic
3	20 min	50	Après-midi — maintien
4	30 min	15	Soir — descente
5	10 min	5	Nuit — creux

Cette séquence reproduit une alternance de périodes de faible activité et de forte sollicitation, afin d'éviter qu'une variation normale de charge soit interprétée comme une dégradation applicative.

8.3 Modes de faute et mécanismes d'injection

Faute / Action	Forme injectée	Endpoint d'injection	Paramètres
latency_creep	Rampe progressive de latence	POST /faults/latency_creep	target_ms, duration_minutes
latency_step	Latence fixe immédiate	POST /faults/latency_step	latency_ms
pool_shrink	Réduction du pool DB	POST /faults/pool_shrink	pool_max
error_burst	Injection de 5xx	POST /faults/error_burst	error_rate (0.0–1.0)
downstream_slow	Ralentissement d'une dépendance	POST /faults/downstream_slow	latency_ms

Faute / Action	Forme injectée	Endpoint d'injection	Paramètres
bad_deploy	Injection latency_step précédée d'un enregistrement de déploiement	POST /deploys + POST /faults/latency_step	—

CHOIX TECHNIQUE

Note sur bad_deploy : dans le fichier `bad_deploy.yaml` actuel, la faute injectée est toujours `latency_step`. Ce n'est pas une contrainte du mécanisme : le runner enregistre le `deploy_event` avant d'exécuter les fautes listées dans le YAML, quel que soit leur type. Le scénario pourrait tout aussi bien utiliser `error_burst` ou `latency_creep`.

Un endpoint `POST /faults/reset` remet tous les états de faute à zéro entre deux scénarios, garantissant l'isolation des injections.

8.4 Scénarios d'injection associés

Les types de fautes précédents sont déclinés en scénarios paramétrés définis dans le scenario-runner. Chaque scénario correspond à une configuration d'injection permettant d'évaluer un comportement particulier du détecteur.

Fichier scénario	Type de faute	Service ciblé	Paramètres clés	Objectif
latency_creep_orders.yaml	latency_creep	orders	target_ms=800, 2 min	Dégradation progressive de latence
latency_creep_slow.yaml	latency_creep	orders	target_ms=400, 3 min	Tester la sensibilité aux dégradations lentes
latency_step_small.yaml	latency_step	orders	100 ms	Tester la limite de détection sur un faible signal
latency_step_large.yaml	latency_step	orders	1000 ms	Tester une dégradation forte
error_burst_low.yaml	error_burst	payments	20 %	Évaluer les faibles pics d'erreurs
error_burst_payments.yaml	error_burst	payments	70 %	Simuler une dégradation importante
error_burst_high.yaml	error_burst	payments	90 %	Tester un incident franc
pool_shrink_orders.yaml	pool_shrink	orders	pool_max=1	Saturation du pool de connexions
pool_shrink_heavy.yaml	pool_shrink	orders	pool_max=1, 120 s	Saturation prolongée

Fichier scénario	Type de faute	Service ciblé	Paramètres clés	Objectif
downstream_slow_payments.yaml	downstream_slow	payments	600 ms	Tester une dégradation en cascade
bad_deploy.yaml	latency_step + deploy event	orders	500 ms, 90 s	Attribution après déploiement
combined_latency_errors.yaml	latency_step + error_burst	orders + payments	400 ms, 40 %	Tester des fautes combinées
quiet_baseline.yaml	Aucune faute	—	—	Mesurer les faux positifs en fonctionnement normal
test_basic.yaml	latency_step + error_burst	orders + payments	300 ms, 50 %	Validation fonctionnelle du runner

8.5 Scénario runner et vérité-terrain

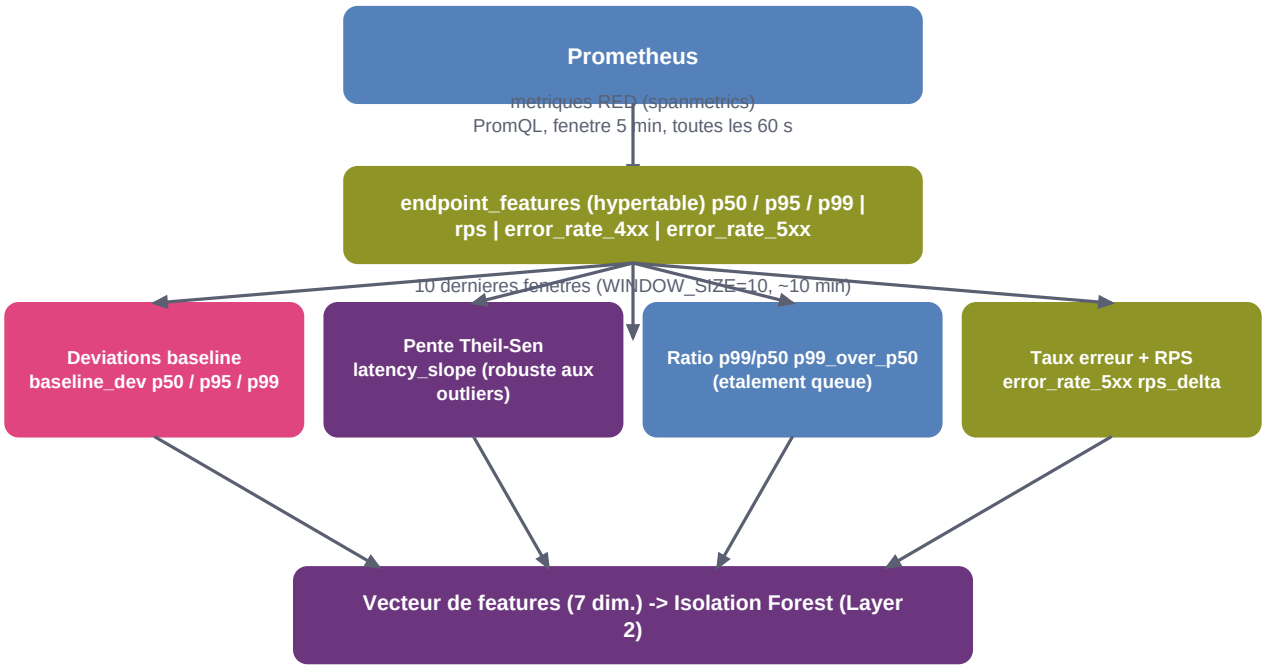
Un CLI orchestre l'exécution des scénarios de **fault injection** décrits dans des fichiers **YAML** déclaratifs, précisant le **type de faute**, le **timing**, la **magnitude** et l'**endpoint cible**. Le **scénario runner** journalise les fenêtres exactes d'injection dans un fichier de **vérité-terrain** (*ground truth*) utilisé par le **pipeline d'évaluation**. Une suite de **14 scénarios** couvrant différents types de fautes et niveaux de sévérité est versionnée avec le dépôt afin d'assurer la **reproductibilité** des campagnes de test.

POINT CLE

POINT CLÉ — Séparation entraînement/évaluation : Les fautes injectées sont utilisées uniquement pour tester le détecteur et sont exclues des données d'entraînement. Le modèle est entraîné uniquement sur des périodes normales afin d'éviter toute fuite de données (*data leakage*) et garantir la validité de l'évaluation.

9 Features et baseline saisonnière

9.1 Fenêtres de features



Pour chaque endpoint et par minute : p50/p95/p99, taux 4xx et 5xx, RPS. Toutes les métriques sont calculées sur une fenêtre glissante de 5 minutes dans Prometheus (lissage intégré, même granularité pour toutes les features). Les features dérivées sont calculées à la lecture par le détecteur :

- **Pente de latence** : ajustement linéaire robuste (Theil-Sen) sur les 10 dernières fenêtres (~10 min de contexte).
- **Ratio p99/p50** : signal d'étalement de la distribution.
- **Deltas court-terme** et **delta de RPS** (pour distinguer une dégradation d'un simple changement de charge).
- **Déviations de baseline** par métrique (voir 9.2) — pour chaque métrique (p50, p95, p99), la valeur mesurée est comparée à la bande saisonnière [p10, p90] du même endpoint à la même heure :
  - Dans la bande [p10, p90] → déviation = **0**
  - Au-dessus de p90 → **(valeur – p90) / (p90 – p10)** *(positif = dégradation)*
  - Sous p10 → **(valeur – p10) / (p90 – p10)** *(négatif = amélioration)*

9.2 Baseline saisonnière conditionnée

Pour chaque endpoint et chaque métrique, on maintient des quantiles conditionnels (p10, p50, p90) indexés sur `(jour_semaine, heure)` sur une fenêtre glissante de 14 jours. On en déduit, par (endpoint, métrique, fenêtre) :

- **expected** = p50 conditionnel ; **band** = intervalle p10–p90 ;

- **baseline_deviation** = position de l'observation vis-à-vis de la bande, **normalisée** (0 dans la bande, distance mise à l'échelle au-delà).

Cold start : si le bucket exact (dow, hour) est absent ou insuffisant (`sample_count < 10`), le fallback agrège min(p10) / avg(p50) / max(p90) sur tous les buckets disponibles du même endpoint et de la même métrique. Aucune couche de clustering n'est nécessaire.

10 | Architecture de détection multicouche

Le cœur du système : trois couches complémentaires, chacune utilisable et démoable seule, dont la sortie est un score d'anomalie calibré par (endpoint, fenêtre).

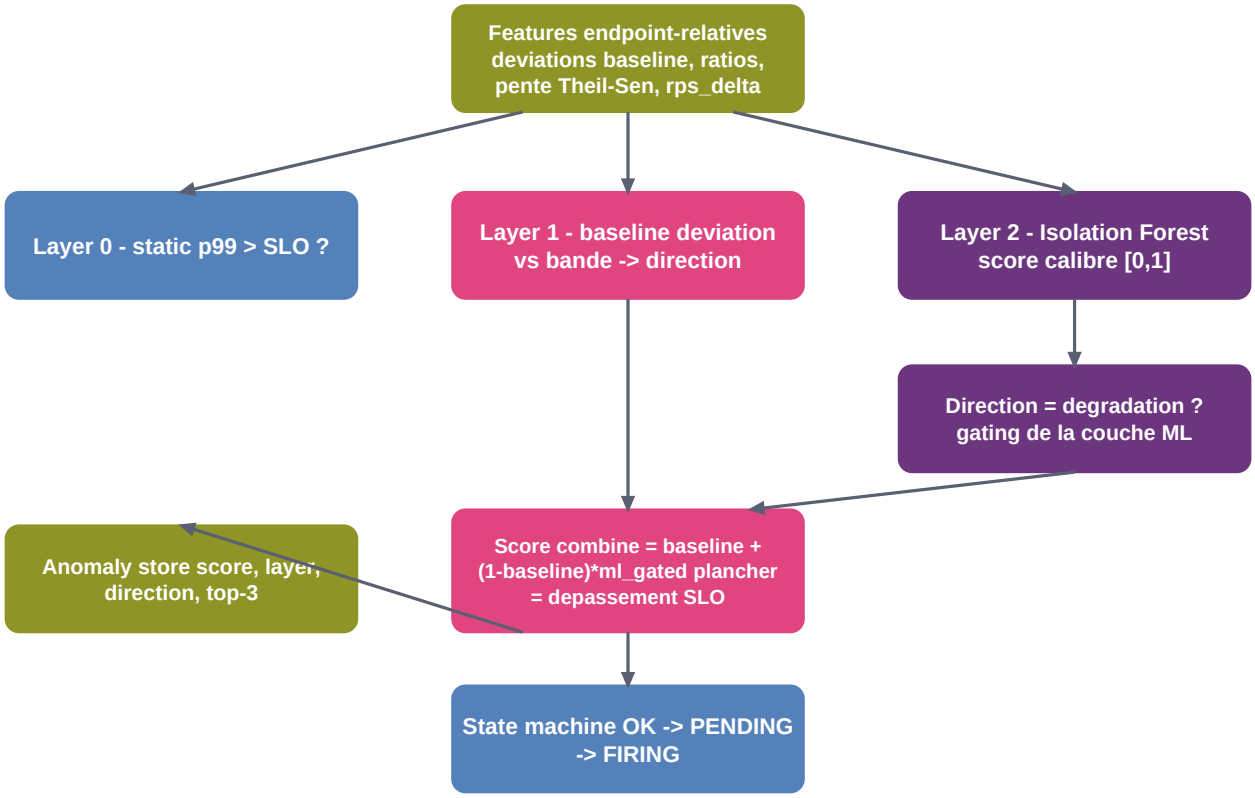


Figure 4 — Pipeline de détection : features endpoint-relatives, couches 0/1/2, direction gating de la couche ML, score combiné, state machine.

10.1 Les trois couches

Couche	Principe	Apport
Layer 0 — static	Seuils SLO classiques (p99 > X sur N fenêtres)	Baseline de comparaison, chemin d'alerte v0
Layer 1 — baseline	Scoring réglé sur baseline_deviation	Direction + plancher, zéro entraînement, saisonnier
Layer 2 — iForest	Isolation Forest non supervisé sur le vecteur de features	Sensibilité au drift multivarié

10.2 Features endpoint-relatives : un modèle global unique

L'Isolation Forest est **un seul modèle global** sur tous les endpoints. Ce n'est valide que parce que les features sont **endpoint-relatives** : déviations de baseline, ratios, pentes, deltas — jamais des niveaux absolus en ms. Alimenter le modèle avec un `p99_ms` brut casserait le modèle global (un endpoint naturellement lent paraîtrait toujours anormal).

10.3 Direction gating

La couche 1 calcule une **direction** (dégradation ou amélioration) à partir de l'observé vs la bande saisonnière. La couche 2 ne contribue à une alerte **que** lorsque la direction est *dégradation*. On n'alerte jamais sur une performance anormalement *bonne*. C'est aussi la mitigation clé contre les faux positifs dus à un simple changement de forme de trafic.

10.4 Combinaison et calibration du score

Le score final combine les couches 1 et 2, avec la couche 2 **gatée par la direction** de la couche 1 :

```
combined = baseline_norm + (1 - baseline_norm) × ml_gated, avec un plancher = dépassement SLO.
```

Le score est calibré sur [0,1] à partir de la distribution du score brut à l'entraînement (p50 → 0, p99 → 1), mise à jour lors de chaque réentraînement nightly. Le déclenchement a lieu sur dépassement SLO **ou** score combiné ≥ **0.60** (seuil tuné). Le score, les contributions par feature (top 3) et la provenance de couche sont écrits dans l'*anomaly store*.

> **CHOIX TECHNIQUE**
>
> **Attribution :** les trois principales features contributrices sont exposées. Pour l'Isolation Forest, via une attribution par profondeur de chemin (path-depth) ; pour la baseline, via les déviations normalisées brutes. C'est ce qui rend l'alerte interprétable par l'opérateur et par le LLM.

11 | Cycle de vie du modèle ML

Le modèle n'est pas figé : il est réentraîné chaque nuit et promu seulement s'il passe un contrôle de sécurité. L'objectif est d'éviter toute dégradation silencieuse.



Figure 5 — Cycle de vie : historique (fautes exclues) -> fit -> artefact horodaté -> sanity gate -> pointeur latest.

11.1 Réentraînement et versionnement

- **Un modèle global**, réajusté *nightly* sur l'historique trailing, avec exclusion stricte des fenêtres d'injection.
- **Artefact horodaté** ; l'artefact précédent est conservé (rollback possible).
- Un pointeur `latest` indique au détecteur quel modèle charger.

11.2 Sanity gate avant promotion

La promotion est **gatée** par un contrôle de dérive de la distribution de score sur une **fenêtre de référence fixe** (test KS entre la distribution des scores du nouveau modèle et celle de l'ancien modèle, évalués sur cette même fenêtre). Si la nouvelle distribution dérive au-delà d'un seuil, la promotion est refusée. S'ajoutent des contrôles de non-régression (recall / faux positifs) et de fraîcheur des données.

POINT CLE

Message défendable : « détecter le drift → réentraîner → valider avant de promouvoir ». Le pipeline ML est autonome *et* sûr : aucune mise à jour ne dégrade le détecteur à l'insu de l'opérateur.

11.3 Le paramètre « évident » qui est inerte

Découverte empirique : le paramètre `contamination` de scikit-learn n'est appliqué qu'à `predict()`, pas à `score_samples()` que l'on calibre — il est donc **inerte** ici et garde à sa valeur par défaut (0.02). Le vrai levier est le **seuil du score combiné**, dont le F1 est maximal à **0.60**.

12 Machine à états d'alerte

Les détections « flottent » (flapping) ; les alertes ne doivent pas. Une machine à états par `(endpoint, signal)` introduit de l'hystérésis à la montée et à la descente.

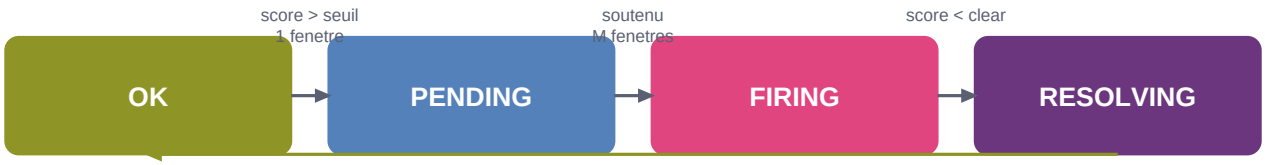


Figure 6 — Machine à états : OK -> PENDING -> FIRING -> RESOLVING -> OK, avec hystérésis M/R et clé de déduplication (endpoint_id, signal_type).

12.1 Transitions et anti-flapping

- **OK** → **PENDING** : score au-dessus du seuil sur 1 fenêtre.
- **PENDING** → **FIRING** : soutenu sur M fenêtres consécutives (hystérésis montée). Dans le détecteur live (`detector.py`, `PENDING_WINDOWS=2`), le compteur démarre à 0 à l'entrée en PENDING et la condition est `pending_count+1 ≥ 2` : il faut **3 cycles consécutifs au total** (1 pour entrer en PENDING + 2 en PENDING) avant de basculer en FIRING. L'évaluateur offline (`evaluate_layered.py`, `M_WINDOWS=2`) déclenche dès le 2e cycle consécutif — les délais rapportés (ex. 124 s median) sont donc **sous-estimés d'environ 60 s** par rapport au comportement live.
- **FIRING** → **RESOLVING** : dès le **1er cycle** dont le score repasse sous le seuil de déclenchement (0.60) ou dont le dépassement SLO est résolu — sans seuil de clear distinct, sans attente supplémentaire.
- **RESOLVING** → **OK** : confirmation sur `RESOLVING_WINDOWS=2` cycles en RESOLVING (hystérésis descente). `set_resolving()` initialise `resolving_count=1` à l'entrée ; la condition `resolving_count ≥ 2` est vérifiée au cycle suivant, puis OK au cycle d'après : il faut **3 cycles clairs au total** pour quitter FIRING (1 pour entrer en RESOLVING + 2 de confirmation) — par symétrie exacte avec la montée.

12.2 Déduplication et économie de coût

Clé de dedup : `(endpoint_id, signal_type)`. Une alerte FIRING est **mise à jour en place** (sévérité, score, attribution) plutôt que renotifiée ; l'escalade ne renotifie que sur augmentation de sévérité. Surtout, **seule la première entrée en état FIRING** (transition PENDING → FIRING) déclenche le moteur de corrélation et l'explication LLM — contrôle du bruit et du coût par construction.

13 Corrélation déploiement et graphe de services

13.1 Fenêtre causale et score d'imputation

Sur une nouvelle alerte FIRING, le moteur cherche les déploiements affectant le service concerné dans une **fenêtre temporelle de corrélation** (par défaut 30 min avant l'*onset*, c.-à-d. l'instant de la transition PENDING → FIRING). Le **score d'imputation** est une fonction transparente de la proximité temporelle (et, à terme, de priors par type d'événement). Il est toujours exposé comme **suspicion, jamais comme cause certaine**.

POINT CLE

Sur le scénario `bad_deploy`, l'alerte nomme le **bon commit/déploiement** en quelques minutes (score de corrélation ~0.9, fenêtre 30 min). C'est le scénario d'attribution bout-en-bout du projet.

13.2 Extension : DAG de dépendances de service

La spec corrèle les déploiements mais ne modélise pas les dépendances inter-services. Or une dégradation du service `orders` se manifeste aussi sur `gateway` et `payments` qui l'appellent. Une extension modélise donc le graphe d'appels réel.

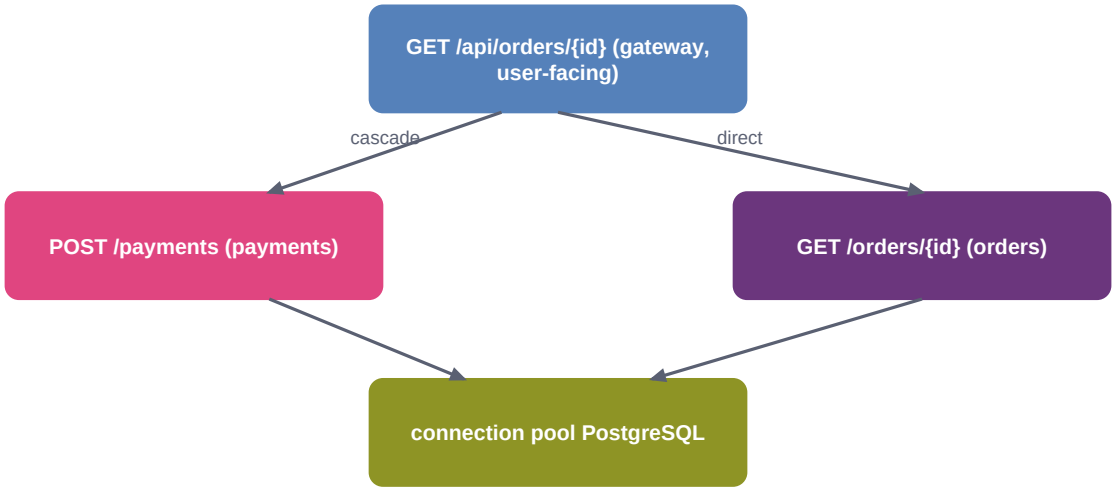


Figure 7 — DAG de dépendances : un même enfant (GET /orders/{id}) est appelé en direct par la gateway et en cascade par payments.

Pourquoi une table de relations, pas une colonne parent ? Un endpoint peut avoir plusieurs parents : le vrai graphe est un **DAG**, pas un arbre. La table `endpoint_relationships` stocke des arêtes dirigées `(parent, child)` avec un `call_type` (*direct* | *cascade*). Un trigger anti-cycle (CTE récursive + CHECK) rejette les cycles au niveau de la base.

13.3 Traversée bornée et priorisation des candidats (P2)

Une requête `WITH RECURSIVE` bornée par `MAX_GRAPH_DEPTH` (défaut 3) remonte les chaînes multi-hop, avec double protection anti-cycles (UNION SQL + garde runtime). Les candidats de cause

racine sont triés par un score de confiance :

```
confidence = min(0.95, anomaly_score × call_type_weight × (1/depth)), avec direct=1.00 et cascade=0.70.
```

CHOIX TECHNIQUE

Invariant dur : l'analyse du graphe est strictement *post-scoring* et best-effort. Toute exception dans `analyze_service_graph()` ne bloque jamais les couches 0/1/2, la state machine ni le chemin d'alerte (`try/except` englobant). Ceci reste une corrélation topologique, pas un remplacement du distributed tracing.

14 | Génération d'explications d'incidents assistée par LLM

Déclenchée une seule fois par transition `PENDING` → `FIRING`, la couche d'explication assemble un contexte structuré : endpoint, métrique(s) en violation, observé vs attendu et magnitude de déviation, top features contributrices, déploiement(s) suspect(s) avec score d'imputation, et historique récent de l'endpoint.

14.1 Contrat de sortie et robustesse

Le contexte est rendu dans un **template de prompt fixe** demandant un résumé de deux phrases, la cause la plus probable cadrée par l'incertitude du score d'imputation, et deux ou trois **premières vérifications concrètes**. La sortie suit un contrat JSON strict (`summary, suspected_cause, checks[]`), **validé avant usage**.

POINT CLE

En cas d'erreur API ou de JSON malformé, l'alerte **bascule sur un template déterministe** : l'alerting ne dépend jamais de la disponibilité du LLM. Priorité 1 du fallback : si des candidats de cause racine existent (DAG), il produit une cause lisible sans LLM.

14.2 Maîtrise du coût et provider

Le coût en tokens est borné par construction : les explications ne partent que sur les transitions d'alerte, jamais par prédiction. L'appel passe par un client **provider-agnostic** (contrat JSON strict + fallback déterministe).

CHOIX TECHNIQUE

Adaptation d'implémentation : le provider retenu est **Gemini** (`google-genai`) plutôt que le client Anthropic de la spec. Le contrat provider-agnostic (JSON strict, fallback) est inchangé.

14.3 Benchmark de sélection du modèle LLM

50 appels exécutés (**2 modèles × 5 scénarios × 5 runs**, `temperature=0`) pour sélectionner le modèle Gemini. Chaque appel est évalué sur : validité JSON du retour, latence API, score de qualité sur rubrique (structure / chiffres / actionnabilité / incertitude, max 12), et erreurs réseau.

Modèle	JSON valid.	Lat. moy.	Lat. p90	Score /12	Erreurs	Verdict
gemini-3.1-flash-lite	100 %	1,6 s	2,6 s	11,6	0	RETENU
gemini-3-flash-preview	76 %	15,3 s	97,2 s	11,4	4 SSL/DNS	Exclu

POINT CLE

gemini-3.1-flash-lite retenu. Score qualité quasi-identique (11,6 vs 11,4/12), mais avantage décisif sur JSON validity (100 % vs 76 %), latence (1,6 s vs 15,3 s moyenne, 2,6 s vs 97,2 s p90), et zéro erreur réseau contre 4 erreurs SSL/DNS.

Le benchmark présenté ici constitue une évaluation comparative utilisée uniquement pour la sélection du modèle LLM. Il ne correspond pas à la métrique finale de qualité des explications présentée dans la section d'évaluation, qui utilise une grille distincte sur 6 points appliquée aux explications générées lors des scénarios d'incident.

15 Méthodologie d'évaluation

La revendication centrale est **relative** : une meilleure couverture de détection et une amélioration de la détection des dégradations progressives, avec un compromis maîtrisé sur le taux de faux positifs. Le protocole la rend directement mesurable.

15.1 Protocole

- Exécuter la suite standard (14 scénarios) avec les détecteurs **statique (Layer 0) et en couches actifs en parallèle**.
- Rapprocher les alertes des fenêtres d'injection : un **vrai positif** est une alerte FIRING sur l'endpoint cible chevauchant (ou dans un délai de grâce) la fenêtre d'injection ; hors fenêtre, c'est un **faux positif**.
- Répéter à plusieurs magnitudes pour produire des **courbes de sensibilité**.

15.2 Métriques

- **Taux de détection** par type de faute et magnitude.
- **Faux positifs par heure** sous charge sans faute (scénarios quiet).
- **Délai de détection médian** (début d'injection → FIRING) et lead time vs seuil statique pour les fautes graduelles.
- **Précision d'attribution** sur bad_deploy : fraction où le déploiement suspect est bien l'injecté.
- **Qualité d'explication** : petit échantillon noté sur rubrique (exactitude des chiffres cités, actionnabilité).

> **CHOIX TECHNIQUE**
>
> L'artefact phare du rapport est le **tableau comparatif (layered vs static par type de faute)**. Toutes les métriques sont exposées dans un tableau de bord Grafana d'évaluation, alimenté par la table `eval_runs`.

16 | Résultats mesurés

Chiffres issus de la campagne d'évaluation historique comprenant 31 fenêtres d'injection valides et des outils d'évaluation dédiés.

71 %	+20 pts	5.00/6	105
Détection (layered)	latency_step vs static	Qualité LLM	Tests automatisés

16.1 Livrable phare : layered vs static

Métrique	Static (Layer 0)	Layered (baseline + ML)
Détection rate (global)	65 %	71 %
Faux positifs / heure	1.13	1.29
Délai de détection médian	—	124 s

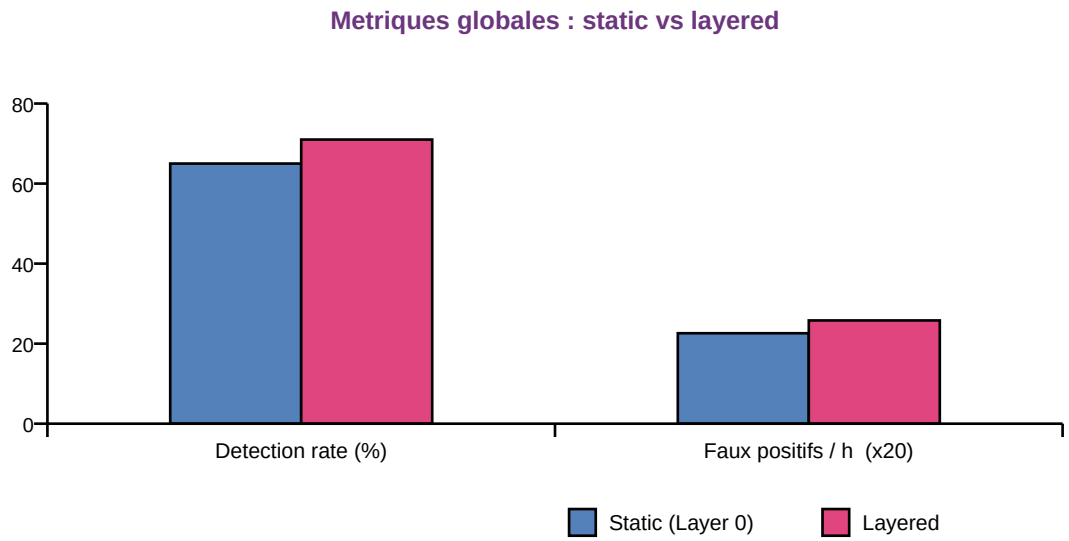


Figure 8 — Métriques globales : le layered détecte plus (71 % vs 65 %) pour un surcoût de faux positifs modeste (FP/h à l'échelle x20).

16.2 Détail par type de faute

Type de faute	Static	Layered	Gain
latency_creep	100 %	100 %	=
downstream_slow	100 %	100 %	=
latency_step	80 %	100 %	+20 pts
error_burst	50 %	50 %	= (signal commun)
pool_shrink	0 %	0 %	= (faute subtile)

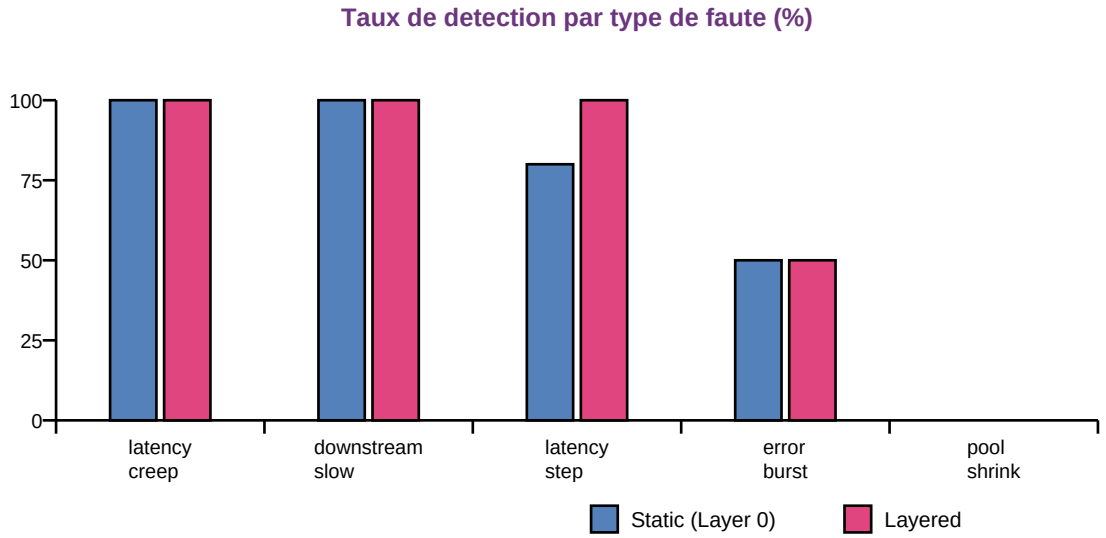


Figure 9 — Taux de détection par type de faute : le layered domine sur latency_step (certains échelons ne franchissent pas le seuil SLO absolu mais dépassent la bande baseline ; le score combiné les détecte).

16.3 Courbe de sensibilité

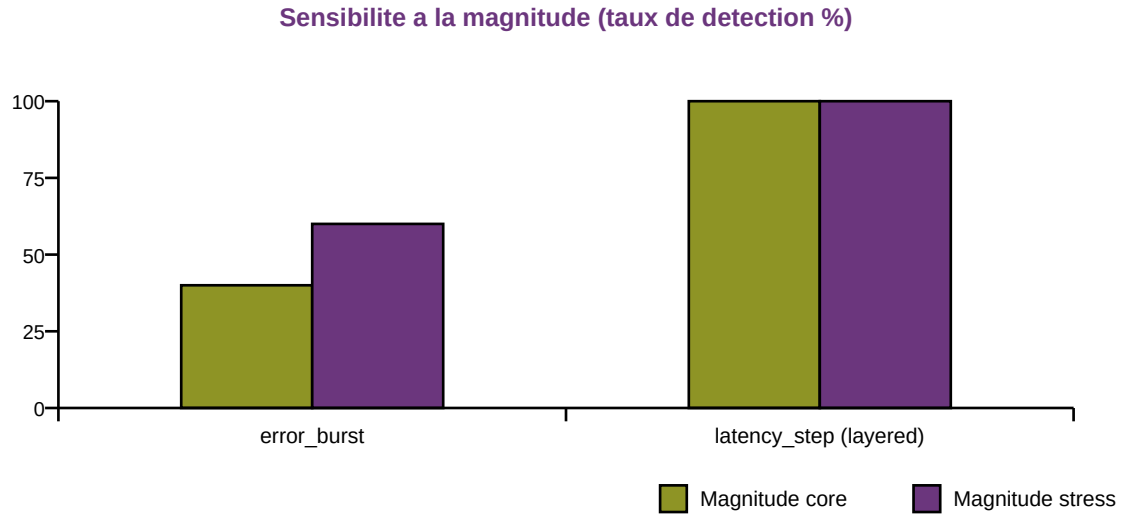


Figure 10 — Le taux de détection croît avec la magnitude (forme attendue) ; l'avantage layered sur latency_step est stable aux deux intensités.

16.4 Autres résultats

Dimension	Résultat
Attribution déploiement	bad_deploy nomme le bon commit en minutes (score ~0.9)
Qualité des explications LLM	5.00 / 6 (structure, chiffres, actionnabilité, incertitude) — évaluation indépendante du benchmark de sélection (§14.3)
Supervisé vs non-supervisé	PR-AUC 0.167 (iForest) vs 0.164 (gradient boosting) : pas d'avantage décisif
Tuning	contamination inerte ; seuil de déclenchement 0.60 = F1 max
Compression TimescaleDB	ratio ~6x (append-only, chunks > 7 j)
Tests / CI	105 tests automatisés, CI GitHub Actions verte

La note de 5,00/6 ne doit pas être interprétée comme une conversion du score 11,6/12 obtenu lors du benchmark de sélection (§14.3). Elle correspond à une évaluation indépendante appliquée aux explications générées par le modèle retenu dans les scénarios d'incident utilisés pour l'évaluation finale.

POINT CLE

Le supervisé ne battant pas le non-supervisé ici (PR-AUC quasi identique), le choix non supervisé est **justifié par la mesure** : il évite la dépendance à des labels d'incidents rares et ne consomme pas le jeu de test.

17 | Sécurité et self-observabilité

17.1 Sécurité

- **Rédaction PII** par denylist d'attributs au Collector (url, ip, identifiant) : aucune donnée de payload ingérée.
- POST /deploys protégé par clé API (X-API-Key) quand la variable `DEPLOY_API_KEY` est configurée ; non authentifié en mode dev.
- API de fault injection bornée au réseau interne, jamais exposée sur l'API plateforme.
- Secrets (Slack, clé LLM) via injection d'environnement, jamais dans le dépôt.

17.2 Self-observabilité

La plateforme se supervise elle-même à travers deux sources distinctes. Le **dashboard Grafana** agrège des métriques Prometheus natives du détecteur (`cassandra_detector_*` : durée de cycle, alertes par état, taux LLM, fraîcheur de scrape) et des requêtes SQL sur TimescaleDB (qualité d'alerte : alertes/jour, durées FIRING, cadence scraper/détecteur). Une sonde de liveness surveille la disponibilité de l'OTel Collector ; ses métriques internes (queue, drops) sont collectées par Prometheus mais ne sont pas encore surfacées dans un panneau dédié. Le **détecteur** expose son endpoint `/metrics` Prometheus natif (port 9101), scrappé directement par Prometheus.

CHOIX TECHNIQUE

Le détecteur expose un endpoint `/metrics` Prometheus natif (durée de cycle p95 ~47 ms, alertes par état, taux LLM, fraîcheur). La dette de stockage est maîtrisée par compression et rétention TimescaleDB (~6x).

18 Limites et travaux futurs

18.1 Limites assumées

- **pool_shrink non détecté** sur le signal p99 aux magnitudes testées : faute de saturation subtile. Piste : intégrer `pool_wait_ms` comme feature optionnelle.
- **Baseline compressée** : la saisonnalité de démo (journée en 2 h) surestime la qualité de baseline vs un vrai cycle diurnal (documenté ; clés period-agnostic).
- **PII en denylist** (allowlist recommandée mais casse spanmetrics — à retravailler).
- **Cold-start** : un endpoint sans baseline neutralise la couche ML (direction indéterminable).

18.2 Travaux futurs

- **Alerte précoce TTD** : extrapolation robuste (Theil-Sen) de la tendance p99 vers le SLO, déjà implémentée en mode advisory, à durcir et surfacer davantage.
- **Priors d'imputation par type de déploiement** (release > migration > config > flag) une fois la colonne `kind` introduite.
- **Propagation par topologie de traces** (distributed tracing) pour dépasser la corrélation topologique actuelle.
- **Features de saturation** généralisées et allowlist PII compatible spanmetrics.

19 | Conclusion

Cassandra démontre qu'une détection **saisonnière et non supervisée** dépasse le seuillage statique sur le critère qui compte — détecter plus et plus tôt les dégradations progressives — avec une augmentation limitée des faux positifs (+0.16 FP/h), correspondant au compromis précision/couverture attendu d'un détecteur plus sensible, et que ce gain est **chiffré** (71 % vs 65 %, +20 points sur latency_step).

Au-delà de la détection, la plateforme **attribue** chaque incident à un déploiement probable (score de suspicion honnête, DAG de services) et l'**explique** en langage naturel via un LLM, avec un fallback déterministe garantissant que l'alerting ne dépend jamais d'un service externe.

L'ensemble est **reproductible** (`docker compose up`), **testé** (105 tests, CI verte) et **observable** (métriques natives, dashboards). Les limites sont assumées et documentées, et les travaux futurs (TTD durci, priors d'imputation, propagation par traces) tracent une suite naturelle.

BILAN DU STAGE

Apports personnels : conception d'un pipeline d'observabilité de bout en bout, mise en œuvre d'une détection ML non supervisée *interprétable*, rigueur d'évaluation (vérité-terrain, comparatif chiffré, honnêteté sur les limites) et pratiques d'ingénierie (Docker, CI, self-observabilité, sécurité).

Références

- OpenTelemetry — spécification et connecteur *spanmetrics* (métriques RED).
- TimescaleDB — hypertables, continuous aggregates, compression et rétention.
- F. T. Liu, K. M. Ting, Z.-H. Zhou, *Isolation Forest*, ICDM 2008.
- scikit-learn — `IsolationForest`, pipelines de preprocessing.
- P. K. Sen, *Estimates of the regression coefficient based on Kendall's tau* (estimateur Theil-Sen), 1968.
- Prometheus — modèle de données et remote-write. Grafana — tableaux de bord.
- Documentation projet : `docs/design-spec.md`, `docs/architecture.md`, `docs/results.md`.